

COS 350 Lecture 21

shell scripting

numbers

tests

functions

pushd & popd

smsh1.c

environment

Bash shell scripting

Variables:

```
PS1="bash> "  
PATH=$PATH:.
```

If statements:

```
if gcc $*  
then  
    a.out  
fi
```

Loops:

```
for name in *  
do  
    echo mailing to $name  
    mail $name < $name  
    sleep 2  
done
```

```
#!/bin/bash
# countdown script
# to demonstrate while loops, numbers, and tests
n=10
while [ $n -gt 0 ]
do
    echo $n
    sleep 1
    let n=n-1
done
echo Blastoff!
```

numeric tests: [val1 -eq val2]
-eq -ge -gt -le -lt -ne

strings tests: [str1 = str2]
= !=

file tests: [-d file]
-d is it a directory
-e does it exist
-f is it a regular file
-r is it readable

Functions:

```
# run emacs in background  
e()  
{  
    emacs $* &  
}
```

Functions:

```
# run emacs in  
background
```

```
e()  
{  
  emacs $* &  
}
```

don't declare arguments
just use: \$1, \$2, ...

run emacs

pass in
ALL
arguments

run in background

Usage:

```
unix> e file.c
```

bash keeps a directory stack so you can switch to a directory and then quickly switch back.

```
bash> pushd /usr/class/cos350
/usr/class/cos350 ~/350/lectures/121
bash>
.
.
.
bash> popd
~/350/lectures/121
bash>
```

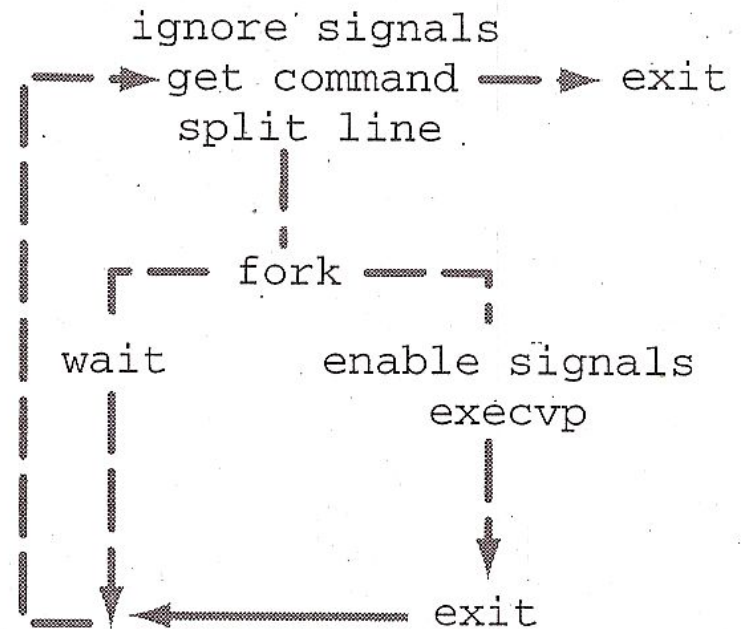
also `pushd` with no argument swaps the top 2 entries on the stack

smsh1.c

```
int main()
{
    char *cmdline, *prompt, **arglist;
    int result;

    prompt = DFL_PROMPT ;
    setup(); // ignore ctrl-C & ctrl-\

    while ( (cmdline = next_cmd(prompt, stdin)) != NULL ) {
        if ( (arglist = splitline(cmdline)) != NULL ) {
            result = execute(arglist);
            freelist(arglist);
        }
        free(cmdline);
    }
    return 0;
}
```



```
void setup()  
/*  
 * purpose: initialize shell  
 * returns: nothing. calls fatal() if trouble  
 */  
{  
    signal(SIGINT, SIG_IGN);  
    signal(SIGQUIT, SIG_IGN);  
}  
  
void fatal(char *s1, char *s2, int n)  
{  
    fprintf(stderr, "Error: %s,%s\n", s1, s2);  
    exit(n);  
}
```



```

int execute(char *argv[])
/*
 * purpose: run a program passing it arguments
 * returns: status returned via wait, or -1 on error
 * errors: -1 on fork() or wait() errors
 */
{
    int    pid ;
    int    child_info = -1;

    if ( argv[0] == NULL )                /* nothing succeeds */
        return 0;

    if ( (pid = fork()) == -1 )
        perror("fork");
    else if ( pid == 0 ) {
        signal(SIGINT, SIG_DFL);
        signal(SIGQUIT, SIG_DFL);
        execvp(argv[0], argv);
        perror("cannot execute command");
        exit(1);
    }
    else {
        if ( wait(&child_info) == -1 )
            perror("wait");
    }
    return child_info;
}

```

// child

// default signal behavior

// parent

```

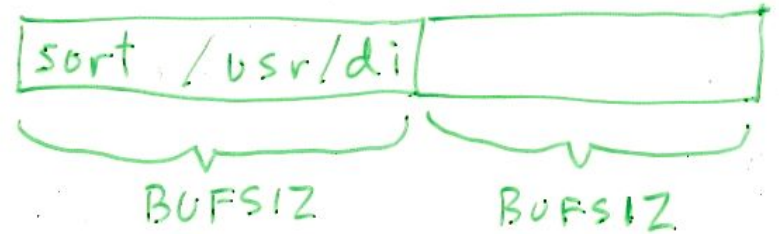
char *next_cmd(char *prompt, FILE *fp)
{
    char *buf;                                // the buffer
    int bufspace = 0, pos = 0, c;

    printf("%s", prompt);                      // prompt user
    while( ( c = getc(fp) ) != EOF ) {
        /* need space? */
        if( pos+1 >= bufspace ) {               /* 1 for \0 */
            if ( bufspace == 0 )                /* y: 1st time */
                buf = emalloc(BUFSIZ);
            else                                 /* or expand */
                buf = erealloc(buf, bufspace+BUFSIZ);
            bufspace += BUFSIZ;                  /* update size */
        }

        /* end of command? */
        if ( c == '\n' )
            break;

        /* no, add to buffer */
        buf[pos++] = c;
    }
    if ( c == EOF && pos == 0 )                 // EOF and no input
        return NULL;                           // say so
    buf[pos] = '\0';
    return buf;
}

```



```
#define is_delim(x) ((x)==' ' || (x)=='\t')
```

```
char ** splitline(char *line)
```

```
/*
```

```
 * purpose: split a line into array of white-space separated  
 *          tokens
```

```
 * returns: a NULL-terminated array of pointers to copies of  
 *          the tokens or NULL if line has no tokens
```

```
 * action: traverse the array, locate strings, make copies
```

```
 * note: strtok() could work, but we may want to add  
 *       quotes later
```

```
*/
```

```
{
```

```
    char *newstr();
```

```
    char **args ;
```

```
    int spots = 0; /* spots in table */
```

```
    int bufspace = 0; /* bytes in table */
```

```
    int argnum = 0; /* slots used */
```

```
    char *cp = line; /* pos in string */
```

```
    char *start;
```

```
    int len;
```

```
    if ( line == NULL )
```

```
        return NULL;
```

```
    // handle special case
```



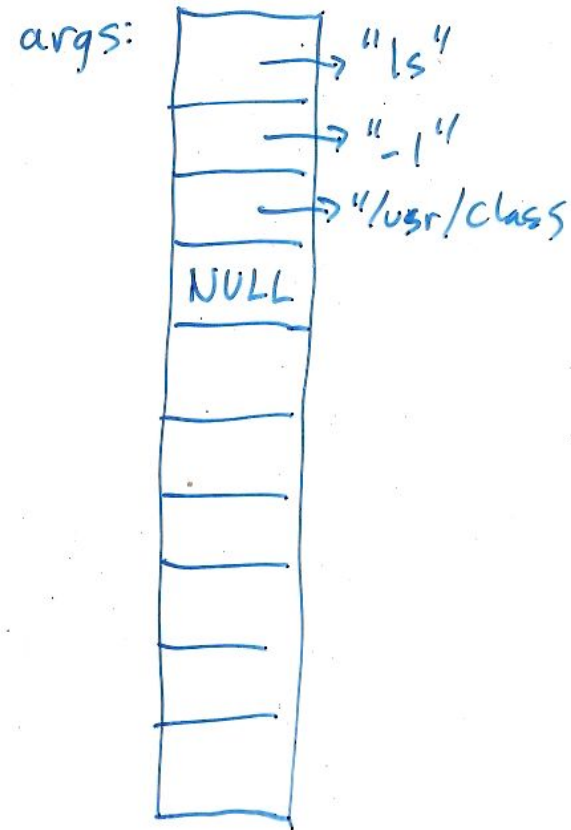
```
args      = emalloc(BUFSIZ);           // initialize array
bufspace = BUFSIZ;
spots     = BUFSIZ/sizeof(char *);
```

ls -l /usr/class/cos350

```
while( *cp != '\0' )
{
    while ( is_delim(*cp) )           // skip leading spaces
        cp++;
    if ( *cp == '\0' )                // quit at end-o-string
        break;
```

/ make sure the array has room (+1 for NULL) */*

```
if ( argnum+1 >= spots ){
    args = erealloc(args,bufspace+BUFSIZ);
    bufspace += BUFSIZ;
    spots += (BUFSIZ/sizeof(char *));
}
```



/ mark start, then find end of word */*

```
start = cp;
len    = 1;
while (*++cp != '\0' && !(is_delim(*cp)) )
    len++;
args[argnum++] = newstr(start, len);
}
```

```
args[argnum] = NULL;
return args;
```

```

/*
 * purpose: constructor for strings
 * returns: a string, never NULL
 */

```

```

char *newstr(char *s, int l)
{
    char *rv = emalloc(l+1);

    rv[l] = '\0';
    strncpy(rv, s, l);
    return rv;
}

```

```

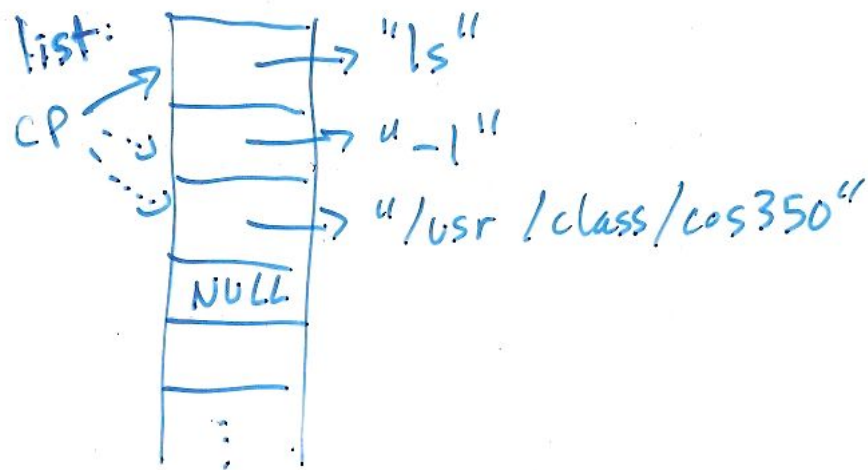
void
freelist(char **list)
/*
 * purpose: free the list returned by splitline
 * returns: nothing
 * action: free all strings in list and then free the list
 */

```

```

{
    char **cp = list;
    while( *cp )
        free(*cp++);
    free(list);
}

```



```
void * emalloc(size_t n)
{
    void *rv ;
    if ( (rv = malloc(n)) == NULL )
        fatal("out of memory", "", 1);
    return rv;
}

void * erealloc(void *p, size_t n)
{
    void *rv;
    if ( (rv = realloc(p,n)) == NULL )
        fatal("realloc() failed", "", 1);
    return rv;
}
```

```
set  
env
```

```
bash> env
```

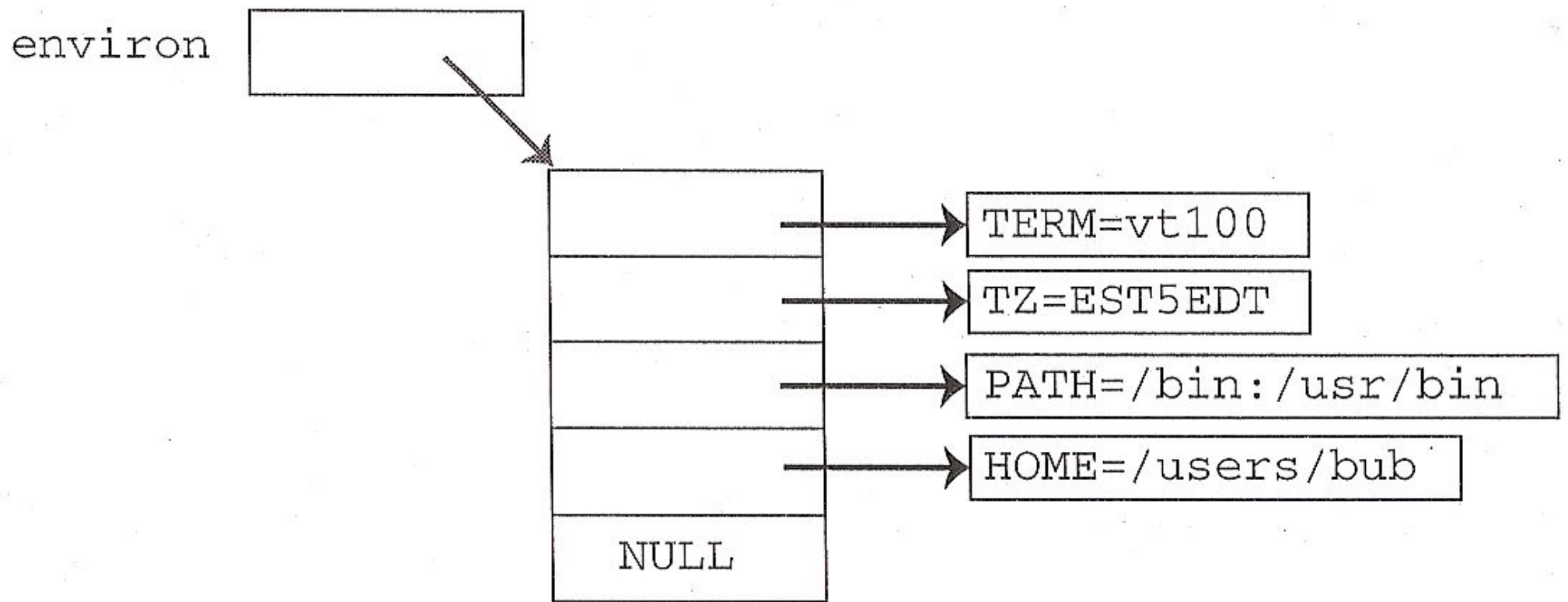


FIGURE 9.6

The environment is an array of pointers to strings.


```
// showenv.c - shows how to read and print the environment
extern char **environ; /* points to the array of strings */

main()
{
    int i;

    for( i = 0 ; environ[i] ; i++ )
        printf("%s\n", environ[i] );
}
```

```
bash> showenv
TERM=xterm
SHELL=/bin/bash
ECLIPSE_HOME=/usr/local/eclipse
XDG_SESSION_COOKIE=cf1ca270b5bcf09cf9f274424a4a1a39-1271125
680.771518-1333275348
SSH_CLIENT=74.78.30.145 1740 22
SSH_TTY=/dev/pts/1
ANT_HOME=/usr/local/apache_ant
```

```
// getenv.c
// demo getting the value of an environment variable
#include <stdio.h>
#include <stdlib.h>

main()
{
    char *s = getenv("HOME");

    if (s == NULL)
        fprintf(stderr, "no match found\n");
    else
        printf("%s\n", s);
}
```

env: HOME=/home/faculty/houser

output: /home/faculty/houser

also:

int setenv(const char *name, const char *value, int overwrite);

0 = don't replace

1 = allow replace

int unsetenv(const char *name);

environment variables get passed to a child process but are not changed in the parent

```
/* changeenv.c - shows how to change the environment
 * note: calls "env" to display its new settings
 */
#include <stdio.h>

extern char ** environ;

main()
{
    char *table[3];                // make our own table

    table[0] = "TERM=vt100";       // fill the table
    table[1] = "HOME=/on/the/range";
    table[2] = 0;

    environ = table;               // point to our table

    execlp("env", "env", NULL);    // exec a program
}
```

What happens?

```
/* changeenv.c - shows how to change the environment  
* note: calls "env" to display its new settings  
*/
```

```
#include <stdio.h>
```

```
extern char ** environ;
```

```
main()
```

```
{
```

```
    char *table[3];                                // make our own table
```

```
    table[0] = "TERM=vt100";                        // fill the table
```

```
    table[1] = "HOME=/on/the/range";
```

```
    table[2] = 0;
```

```
    environ = table;                                // point to our table
```

```
    execlp("env", "env", NULL);                    // exec a program
```

```
}
```

```
bash> changeenv
```

```
TERM=vt100
```

```
HOME=/on/the/range
```

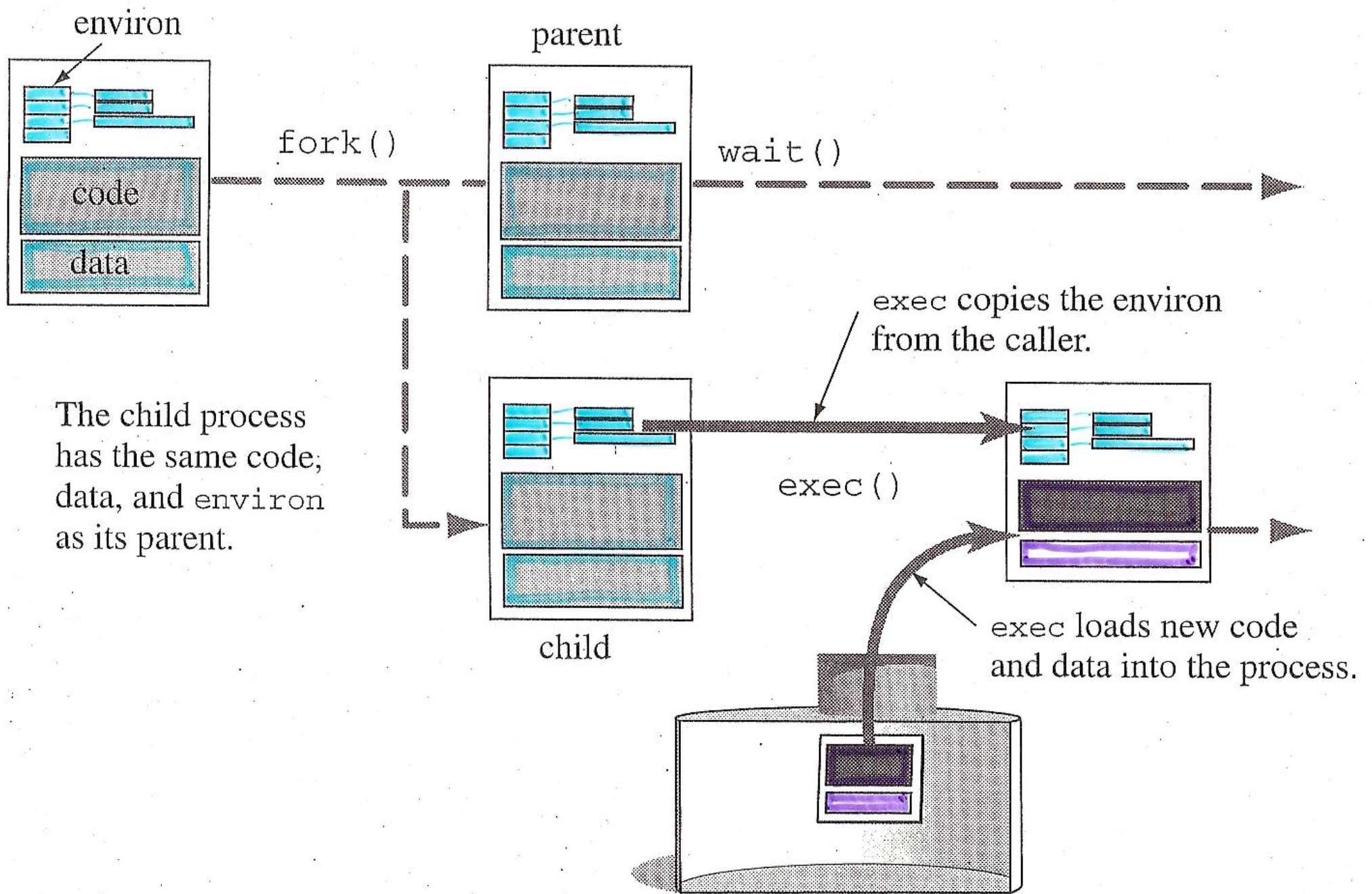



FIGURE 9.7

Strings in environ are copied by `exec()`.